
Madison-RL

Technical Implementation Guide

An exhaustive technical breakdown of the Reinforcement Learning architecture, experimental design, and algorithmic foundations.

Abhinav Kumar Piyush

NUID: 002038671

Spring 2026

1. System Architecture

Madison-RL implements a **Hierarchical Orchestration** framework where a high-level orchestrator dispatches a team of low-level specialist agents to solve complex, multi-modal intelligence tasks.

1.1 Architectural Data Flow

The system follows this pipeline: the Gymnasium Environment generates a 16-d task embedding, producing a 25-d observation vector. A Learned Orchestrator (PPO/DQN policy network) selects a discrete action from 5 options. The Specialist Layer (IPPO) receives the specialist ID, makes an effort selection, and returns quality gain, cost, and confidence. A shared team reward feeds back to both orchestrator and specialist policies.

2. Markov Decision Process (MDP)

The system is formulated as a discrete-time, finite-horizon MDP $\langle S, A, P, R, \gamma \rangle$.

2.1 State Space (S)

The observation space is a 25-dimensional continuous vector s in $\mathbb{R}^{25}$ containing:

Indices	Component	Range	Description
0-15	Task Embedding	$[-1, 1]$	16-d latent representation of task requirements.
16	Partial Quality	$[0, 1]$	Cumulative quality score achieved so far.
17	Norm. Step	$[0, 1]$	Current step / time budget.
18	Norm. Cost	$[0, 1]$	Incurred cost / cost ceiling.
19-22	Usage Mask	$\{0, 1\}$	Binary flags for utilized specialists.
23	Last Conf.	$[0, 1]$	Confidence from previous dispatch.
24	Last Gain	$[0, 1]$	Quality increment from last step.

2.2 Action Space (A)

A **Discrete(5)** action space representing:

- **0**: Dispatch Researcher
- **1**: Dispatch Analyst
- **2**: Dispatch Synthesizer
- **3**: Dispatch Validator
- **4**: FINISH (Manually terminate episode)

2.3 Reward Engineering (R)

The project uses a hybrid reward signal designed to solve the sparse terminal reward problem while optimizing efficiency:

Dense Shaping Reward:

$$r_{\text{step}} = (0.5 \times \text{Delta_Quality}) - (0.02 \times \text{Cost})$$

The 0.5 multiplier incentivizes steady progress; the -0.02 cost penalty prevents loops and encourages efficient dispatch.

Sparse Terminal Reward:

- Success (Quality $\geq$ Threshold): $r_{\text{term}} = (10.0 \times \text{Quality}) + (1.5 \times \text{Budget_rem} / \text{Budget_total})$
- Failure (Quality $<$ Threshold): $r_{\text{term}} = -2.0$

3. Implemented RL Categories (5-Rubric Requirement)

Madison-RL implements all five categories requested by the project rubric:

3.1 Category 1: Value-Based Learning (DQN)

File: train_dqn.py

Deep Q-Learning with a 50k-step Replay Buffer and hard Target Network updates. Uses Hard Target Update (tau=1.0) every 500 steps for maximum stability in the discrete environment.

3.2 Category 2: Policy Gradient (PPO)

File: train_orchestrator.py

Proximal Policy Optimization with GAE (lambda=0.95). Optimizes the clipped surrogate loss with an Entropy bonus (0.01) to maintain diverse specialization early in training.

3.3 Category 3: Multi-Agent RL (IPPO)

File: train_marl.py

Independent PPO for specialists co-adapting with the orchestrator. Each specialist receives the exact same reward (Shared Reward), creating a fully cooperative multi-agent system.

3.4 Category 4: Exploration Strategies

File: train_linucb.py / IntelligenceTaskEnv

Contextual Bandit (LinUCB) for one-shot routing comparison. Count-based Novelty with intrinsic bonus $r_i = \text{beta} / \sqrt{n}$ for rare (Task-Bucket, Action) pairs.

3.5 Category 5: Transfer Learning

File: train_transfer.py

PPO pre-trained on Easy tasks then fine-tuned on Hard tasks. Achieves 2.2x speedup in adaptation compared to training from scratch.

4. Hyperparameter Registry

Parameter	PPO Value	DQN Value	IPPO Value
Learning Rate	3e-4	5e-4	2e-2 (Linear)
Batch Size	64	64	32 (Episodes)
Gamma	0.98	0.98	1.0 (Undiscounted)
Hidden Layers	[64, 64]	[64, 64]	[Linear]
Exploration	Entropy: 0.01	eps_final: 0.05	Stochastic Policy
Total Steps	150,000	80,000	6,000 Episodes

5. Specialist Response Modeling

Specialists are modeled with high-fidelity Expertise Prototypes:

- **Capability Match:** Quality gain depends on cosine similarity between the task's required capability vector and the specialist's expertise vector.
- **Effort Potency:**

Effort Level	Cost	Potency
Low	0.8	0.4
Medium	2.2	0.7
High	4.5	1.1

- **Diminishing Returns:** Gain is multiplied by (1.0 - Current_Quality). As quality approaches 100%, gaining the final 1% becomes exponentially harder.

6. Neural Architecture Specification

All deep neural networks (PPO and DQN) share a unified MLP architecture:

Layer	Configuration	Description
Input	(25,)	Normalized observation vector.
Hidden 1	Dense(64)	Linear + ReLU activation.
Hidden 2	Dense(64)	Linear + ReLU activation.

Layer	Configuration	Description
Actor Output	Dense(5)	Softmax (PPO) / Linear (DQN Q-values).
Critic Output	Dense(1)	Value function V(s) for PPO baselining.

Note: For MARL specialists, a Linear Softmax policy is used to minimize computational overhead during co-adaptation, initialized with a strong Medium-Effort bias for stable warm-starts.

7. Mathematical Formulations

7.1 PPO Clipped Objective

The orchestrator optimizes the clipped surrogate objective for small, stable policy updates:

$$L_{CLIP}(\theta) = E_t [\min(r_t(\theta) * A_{hat}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon) * A_{hat}_t)]$$

Where $r_t(\theta)$ is the probability ratio and A_{hat}_t is the Generalized Advantage Estimate.

7.2 IPPO Specialist Objective

$$\text{grad}_{\theta_i} J(\theta_i) = E [\text{grad}_{\theta_i} \log \pi_i(a|s) * (R - b)]$$

Where b is a running baseline of the team reward to reduce variance.

7.3 LinUCB Action Selection

$$a_t = \text{argmax}_a (\theta_{hat}_a^T x_t + \alpha * \sqrt{ x_t^T A_a^{-1} x_t })$$

Where α controls exploration vs. exploitation.

8. Environment Mechanics: The Quality Surface

The quality gain for specialist s on task T is:

$$\Delta Q = \text{Match}(s, T) \times \text{Effort}(s) \times (1.0 - Q_{current})$$

- **Diminishing Returns:** The $(1.0 - Q_{current})$ term makes the state space non-linear. As the solution nears 100%, gaining the final 1% is exponentially harder than the first.
- **Task Drifting:** Each specialist call adds noise to task embeddings, simulating how real-world intelligence tasks evolve as more data is gathered.

9. Reproducibility Guide

To reproduce the 99% success rate results, follow this exact pipeline:

1. Baseline Training:

```
python -m madison_rl.training.train_all --seeds 0 1 2 3 4
```

2. MARL Specialist Tuning:

```
python -m madison_rl.training.train_marl --seed 0 --episodes 5000
```

3. Transfer Learning Benchmark:

```
python -m madison_rl.training.train_transfer --seed 0
```

4. Verification:

```
python -m madison_rl.eval.run_experiments  
python -m madison_rl.eval.stats
```

10. Rubric Category Compliance Matrix

Rubric ID	Feature	Implementation File
Cat 1	Value-Based RL	train_dqn.py
Cat 2	Policy Gradient	train_orchestrator.py
Cat 3	Multi-Agent RL	train_marl.py
Cat 4	Exploration	train_intrinsic.py
Cat 5	Transfer Learning	train_transfer.py

11. Custom Tools and Debugging

The **Trajectory Debugger** (debugger.py) visualizes internal RL agent state:

- **GAE Advantages:** Highlights which specialist the PPO agent credited with rescuing the task.
- **Value Estimates:** Shows how the agent's confidence in task success grows over time.
- **Credit Assignment:** Visualizes how shared reward is attributed to individual specialist contributions.